

ENRICO FIORI FERRAZ

POSTECH

IA PARA DEVS

FINE-TUNING E RAG PARA DOCUMENTOS

AULA 03

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?.....	13
REFERÊNCIAS	14

EMSE

O QUE VEM POR AÍ?

Por vezes não basta somente construir uma inteligência artificial que processe linguagem natural. E se for necessário que o modelo retorne a fonte que a IA utilizou para responder sobre determinado tópico? É aqui que entra o RAG, ou Retrieval Augmented Generation. Esse método inovador combina o poder dos modelos de linguagem baseados em transformadores com ferramentas de recuperação de informação e fonte, criando uma abordagem híbrida que potencializa as possibilidades de modelos de geração de texto.

O RAG agrega valor ao output fornecido pela IA, buscando informações em tempo real para embasar o retorno e aumentar sua relevância e precisão do texto gerado, mas também abre novas possibilidades para aplicações práticas, desde assistentes virtuais até ferramentas de pesquisa avançada.

Nesta aula, vamos aprender como implementar RAG para que a resposta de nosso modelo retorne o conteúdo e a sua fonte (nesse caso, o link da notícia).

HANDS ON

Considere como uma IA com RAG pode agregar valor em uma empresa jornalística, por exemplo. Se é solicitado a um analista de dados dessa empresa que retorne todas as matérias já publicadas sobre determinado tema, será preciso utilizar alguma ferramenta de consulta no banco de dados e montar queries para ter o retorno desejado. Mas ao utilizar RAG, o próprio jornalista ou editor pode interagir com uma IA e solicitar diretamente os links das matérias sobre aquele tópico. Essa técnica pode ser utilizada nos mais diversos domínios, como escritórios de advocacia ou no meio acadêmico.

Então, vamos construir uma aplicação de IA utilizando RAG, passando pela coleta dos dados, criação da base de dados vetorizada (onde vamos guardar nossas notícias e as respectivas fontes) e realizando testes para verificar se a resposta retorna não apenas o contexto, mas também a fonte utilizada. Assista os vídeos da aula e retorne aqui para aprofundar seus conhecimentos!

SAIBA MAIS

Pode ser, que durante os vídeos, você quis saber mais sobre as palavras-chave “RAG”, “LangChain” ou “Base de Dados Vetorizadas”. Abaixo, desenvolvemos mais esses temas, mas por serem tecnologias que são atualizadas rapidamente, sugiro que você faça as suas próprias leituras. Busque na web documentações, artigos e livros (além dos fornecidos aqui) que tratem sobre esses assuntos.

RAG - Retrieval Augmented Generation

O Retrieval Augmented Generation, ou RAG, é uma técnica na área de Processamento de Linguagem Natural (NLP) que combina métodos de recuperação de informação (retrieval) com geração de texto em um modelo LLM. A ideia central é enriquecer a capacidade de um modelo de linguagem em gerar respostas mais precisas e informativas, utilizando informações extraídas de um grande conjunto de dados externo.

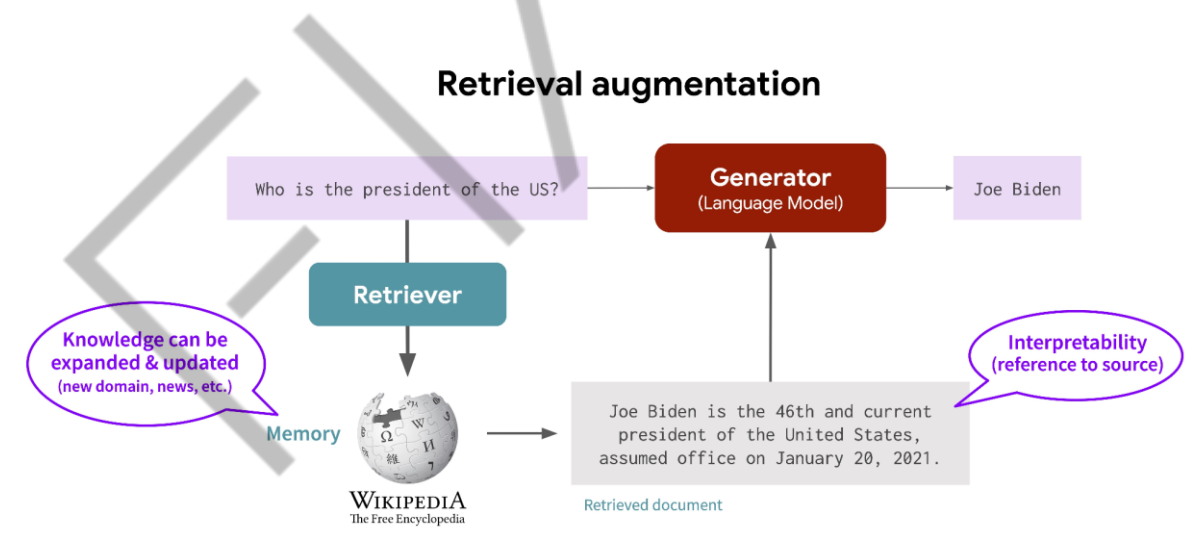


Figura 1 - Exemplo de aplicação do RAG
Fonte: [Stanford](#) (2023)

Vamos ver como o RAG funciona e quais são suas principais características.

Componentes do RAG

Retriever: a primeira etapa em um sistema que utiliza RAG envolve um "retriever" que é responsável por buscar informações relevantes a partir de uma base de dados externa, que pode ser um corpus de documentos, uma base de dados estruturada ou até mesmo a internet. Esse componente usa a consulta ou parte do texto de entrada para encontrar trechos relevantes que possam ajudar a gerar uma resposta mais completa e mais precisa.

Generator: após a recuperação de informações relevantes pelo retriever, o componente de geração, geralmente um modelo de linguagem avançado como Llama, MINSTREL, GPT ou BERT, utiliza essas informações para gerar uma resposta. O generator é capaz de integrar os dados recuperados de várias maneiras, desde a concatenação simples até métodos mais complexos que fundem as informações no processo de geração.

Funcionamento do RAG

O funcionamento do RAG pode ser dividido em etapas:

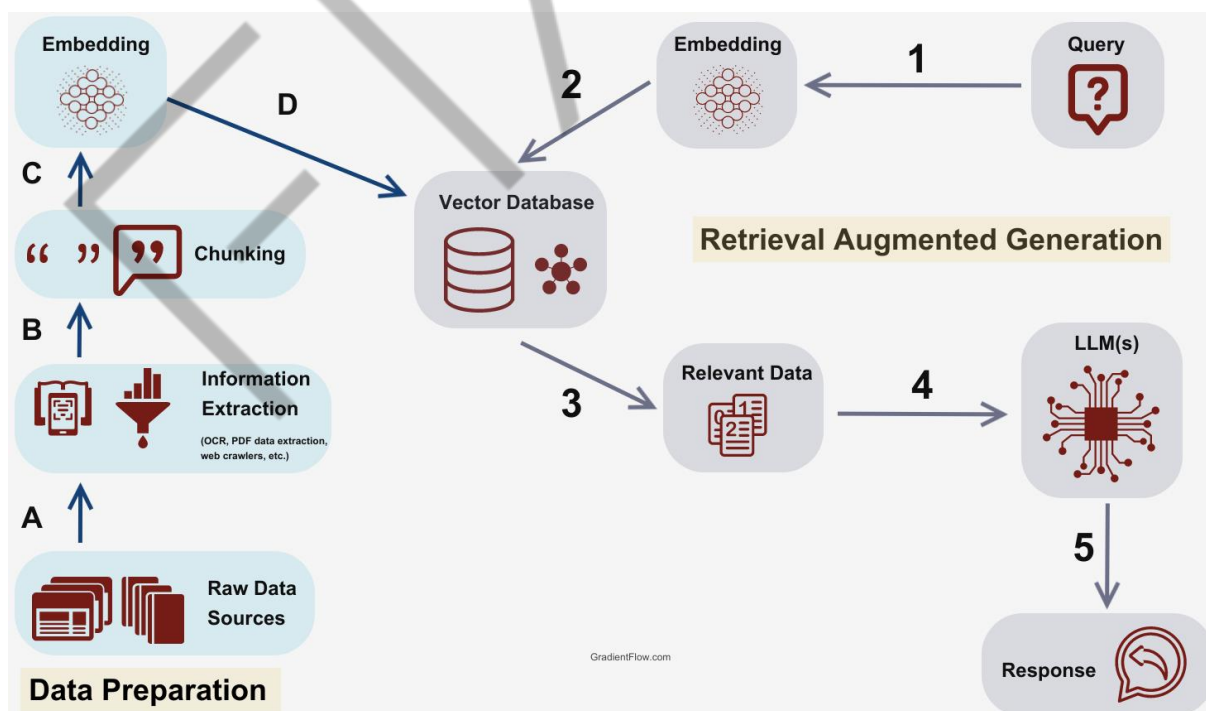


Figura 2 - Ilustração do funcionamento do RAG
Fonte: [Gradient Flow](#) (2023)

- Input: o sistema recebe uma entrada, que pode ser uma pergunta ou um prompt que necessita de uma expansão ou resposta.
- Retrieval: o retriever busca no dataset externo informações pertinentes à entrada. Isso é feito usando técnicas de similaridade de texto, como vetores semânticos ou hash de busca.
- Geração da Resposta: as informações recuperadas são passadas ao modelo gerador, que as utiliza para formular uma resposta. Durante essa etapa, o modelo pode ponderar diferentes partes das informações recuperadas, baseando-se em sua relevância para a entrada.
- Output: o texto gerado é fornecido como output, que pode ser uma resposta direta a uma pergunta, a geração de conteúdo para um tópico específico, entre outros. A depender do contexto dado ao modelo, ele pode ou não retornar a fonte utilizada para compor a resposta.

Embora o RAG ofereça vantagens significativas, como precisão e relevância contextual, aplicações de IA que utilizam essa ferramenta enfrentam desafios, como a escalabilidade na gestão de grandes volumes de dados, a latência na recuperação de informações que pode atrasar respostas e a complexidade na integração eficaz de dados recuperados, necessitando de técnicas avançadas de modelagem e treinamento. Assim, cabe a engenheiros(as) ou desenvolvedores(as) do modelo ponderar se seu programa necessita utilizar o RAG.

Lembre-se de buscar sempre a melhor ferramenta para o seu problema.

LangChain

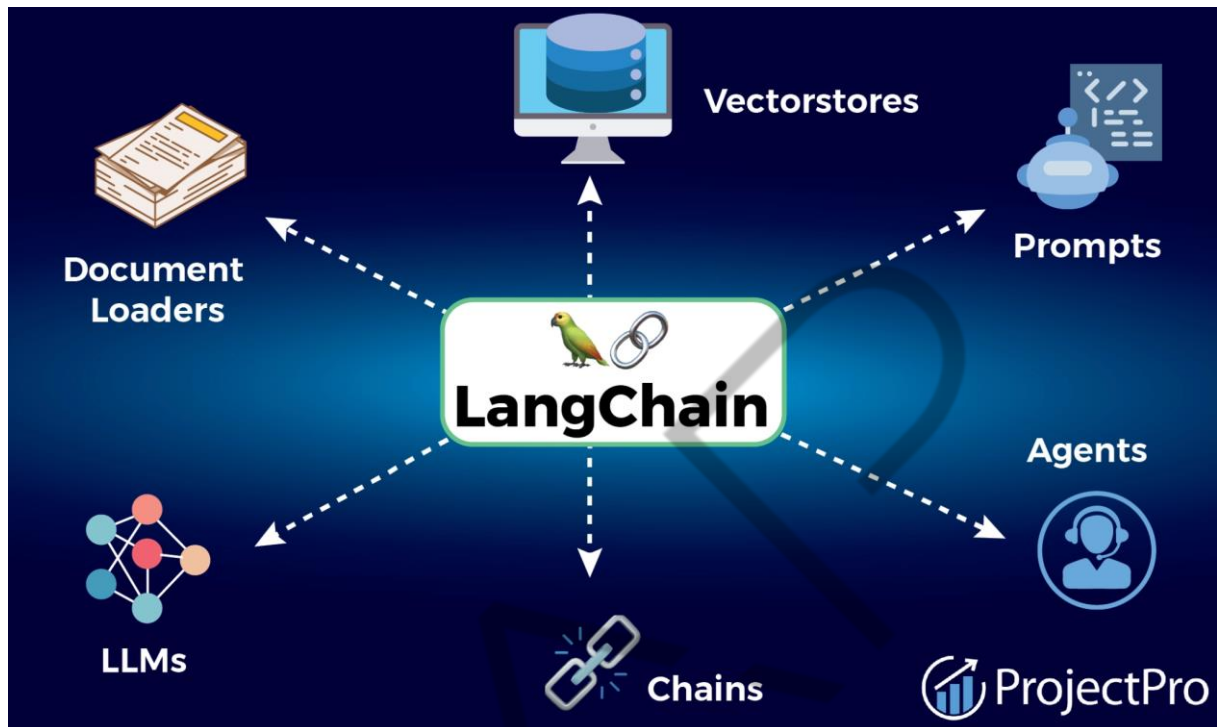


Figura 3 - Aplicações do Framework LangChain
Fonte: [ProjectPro](#) (2024)

LangChain é um framework de desenvolvimento focado em IA que simplifica a criação de aplicações impulsionadas por modelos de linguagem generativa. A partir de uma abordagem modular, LangChain oferece um conjunto de componentes e integrações que facilitam a construção de aplicações de inteligência artificial contextuais e interativas. Recomendo muita a leitura da introdução sobre o LangChain [aqui](#).

Características do LangChain

O LangChain é projetado para ser modular, permitindo que pessoas desenvolvedoras utilizem blocos de construção pré-fabricados ou criem os seus próprios, facilitando a personalização e expansão das funcionalidades da aplicação.

Integração com LLMs: facilita a integração de aplicações com grandes modelos de linguagem, permitindo que eles sejam usados para entender e responder a consultas em linguagem natural.

Abstrações Flexíveis: LangChain fornece abstrações flexíveis que são ideais para o desenvolvimento de aplicações que necessitam de capacidade de raciocínio baseadas em IA, como chatbots e assistentes virtuais.

Ferramentas e Templates: o framework oferece uma variedade de ferramentas e templates que aceleram o desenvolvimento e reduzem a necessidade de codificação detalhada, tornando a tecnologia mais acessível a pessoas desenvolvedoras com diferentes níveis de habilidade técnica.

Comunidade e Suporte: outro ponto positivo do LangChain é a comunidade ativa e que oferece suporte para quem desenvolve por meio de documentação detalhada e tutoriais, proporcionando um ecossistema de usuários e contribuidores que se ajudam e instigam mudanças e alterações para uma melhor experiência no uso da solução.

Utilizações do LangChain

- **Desenvolvimento de Chatbots e Agentes Virtuais:** ideal para criar sistemas que respondem a perguntas e interagem com os usuários em tempo real, usando informações contextualizadas.
- **Ferramentas de Análise de Dados:** pode ser usado para desenvolver interfaces que permitem aos usuários consultar e interagir com grandes volumes de dados de maneira intuitiva.
- **Integração com Outras APIs e Dados Externos:** permite a criação de agentes que podem acessar e manipular informações de várias fontes externas, integrando-se a APIs, bancos de dados e outras plataformas.

LangChain é particularmente útil para pessoas desenvolvedoras que desejam explorar as possibilidades dos modelos de linguagem avançados, integrando-os em

aplicações avançadas que exigem interações em linguagem natural e capacidades de raciocínio contextual.

Base de dados vetorizada - VectorDB

As bases de dados vetorizadas, frequentemente referidas como "vetorização de dados" ou "espaços de incorporação vetorizada", são um conceito crucial no processamento de linguagem natural (PLN) e em outras áreas de aprendizado de máquina. Esse método envolve a transformação de dados brutos, como texto, imagens ou sons, em vetores de características numéricas que representam de maneira eficiente a essência dos dados originais. Aqui estão alguns aspectos importantes sobre as bases de dados vetorizadas:

O Que São Bases de Dados Vetorizadas?

As bases de dados vetorizadas são conjuntos de vetores numéricos onde cada vetor representa um item de dados (como um documento, imagem ou palavra) em um espaço multidimensional. Criamos esses vetores usando alguns métodos sofisticados que descobrem o que é importante nos dados. É como se você tivesse um monte de palavras ou outras coisas e você pudesse transformá-las em números que mostrassem o quão semelhantes ou diferentes elas são em significado e estrutura.

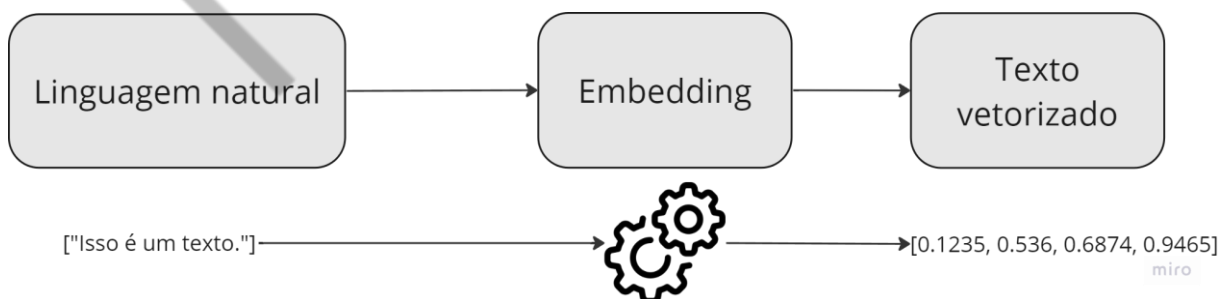


Figura 4 - Ilustração da transformação de texto em vetores
 Fonte: elaborado pelo autor (2024)

Técnicas de Vetorização

TF-IDF (Term Frequency-Inverse Document Frequency): uma técnica que avalia a importância de uma palavra em relação a um conjunto de documentos, considerando a frequência do termo no documento e a frequência inversa do termo em todos os documentos.

Word Embeddings (Incorporação de Palavras): modelos como Word2Vec, GloVe e FastText geram vetores de palavras que capturam contextos semânticos e relacionamentos entre palavras baseados em como elas ocorrem em grandes textos.

Sentence Embeddings (Incorporação de Sentenças): similar aos word embeddings, mas é adaptado para sentenças ou parágrafos inteiros. Modelos como Sentence-BERT ou Universal Sentence Encoder são exemplos de funções que utilizam esse tipo de embedding.

Vantagens das Bases de Dados Vetorizadas

- **Eficiência:** a vetorização reduz a dimensionalidade dos dados, facilitando processamentos e análises rápidas.
- **Precisão:** vetores podem capturar nuances e detalhes dos dados, melhorando a precisão das previsões e análises.
- **Flexibilidade:** podem ser aplicados em diferentes tipos de dados e para variados objetivos analíticos.

Graças à vetorização, o hardware consegue realizar os cálculos necessários para encontrar similaridades e relações entre textos, documentos e outras formas de linguagem natural, o que seria impossível de compreender em sua forma “humana”. Com isso é possível processar de forma eficiente e rápida os dados imputados no modelo e, no caso do RAG, o modelo consegue buscar na base de dados quais são os vetores (o texto convertido) que se referem sobre o tópico imputado pelo usuário, proporcionando uma resposta mais precisa e completa.

Contudo, vetores de alta dimensionalidade podem ser difíceis de interpretar em termos humanos, o que pode ocasionar alucinações e respostas erradas. Outro fator importante é qualidade dos dados, uma vez que a eficácia dos vetores depende fortemente da qualidade e da representatividade dos dados originais.

EMENDAS

O QUE VOCÊ VIU NESTA AULA?

Na aula final dessa disciplina, criamos nossa base de dados vetorizadas para armazenar nossas notícias e suas fontes. Utilizamos o framework LangChain para carregar os textos diretamente a partir do link, sem precisar extrair os dados via script e, posteriormente, limpamos esses dados usando BeautifulSoup (como fizemos na primeira aula) e, por fim, desenvolvemos um chatbot utilizando um RAG que nos responde com base em um contexto específico, em nosso caso, as notícias relacionadas à pergunta do usuário.

Agora é com você! Coloque a mão na massa e, com os conhecimentos adquiridos ao longo das aulas, faça o desafio da disciplina e tente criar um modelo treinado por você. Se tiver dúvidas, poderá retornar às aulas e aos materiais disponibilizados e revisar os assuntos tratados.

Muito obrigado pela atenção e dedicação. Até a próxima!

REFERÊNCIAS

HAN, Y.; LIU, C.; WANG, P. **A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge.** Disponível em: <<https://arxiv.org/abs/2310.11703>>. Acesso em: 28 de maio de 2024.

INTRODUCTION. **LangChain.** Disponível em: <https://python.langchain.com/docs/get_started/introduction/>. Acesso em: 28 de maio de 2024.

RAG (Retrieval-Augmented Generation). **LangChain4j.** Disponível em: <<https://docs.langchain4j.dev/tutorials/rag/>>. Acesso em: 28 de maio de 2024.

ZHAO, P. et al. **Retrieval-Augmented Generation for AI-Generated Content: A Survey.** Disponível em: <<https://arxiv.org/abs/2402.19473>>. Acesso em: 28 de maio de 2024.

PALAVRAS-CHAVE

Retrieval Augmented Generation. LangChain. Base de dados vetorizada.

EMEND



POSTECH